Pre-Read: React Component Lifecycle + `useEffect` (Mounting, Updating, Unmounting, Practical Uses)

1. Why lifecycle matters

React components don't just "show once." They:

- appear on screen (**mount**)
- re-render when state/props change (**update**)
- disappear (**unmount**)

`useEffect` is how you run **side-effects** at the right time.

What are side effects?

A **side effect** is **anything your component does besides calculating and showing UI**.

React wants components to be like this:

- take some data
- return what the screen should look like

That part is clean and predictable.

A **side effect** is when the component also does something **outside itself**, like:

- calling an API
- setting a timer
- changing the browser title
- listening for window resize
- saving to localStorage
- manually touching the DOM

Did you know?
Without cleanup, your app can leak memory (like intervals continuing even after the UI is gone).

A) Lifecycle in simple words

2. The 3 phases

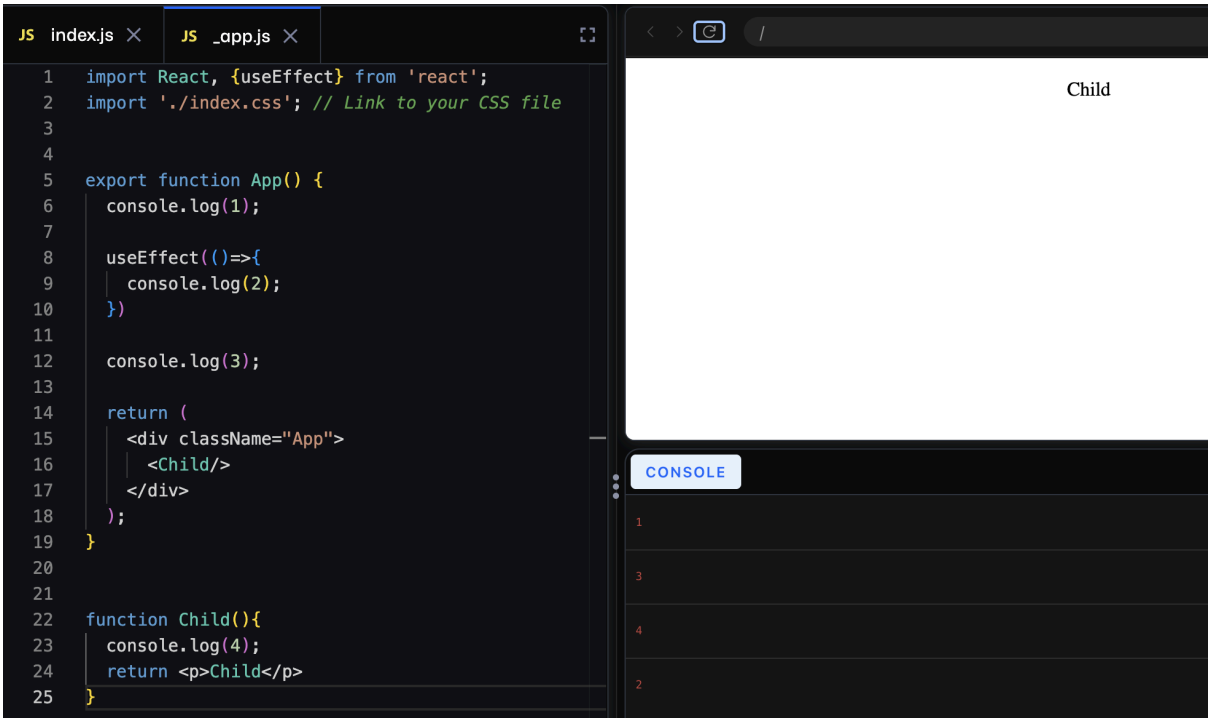
1. **Mounting** → component is created + added to DOM (first time)
2. **Updating** → component re-renders because state/props changed
3. **Unmounting** → component is removed from DOM

B) What is `useEffect`?

`useEffect` is a React hook that runs code **after render**.

```
import { useEffect } from "react";

useEffect(() => {
  // code runs after render
});
```



If you see the order of the console.log‘s. First all code in the **App** is run except for useEffect confirmed by printing **1 and 3**. Then the UI is shown as the Child is also called and **4** is printed. Then in the end the useEffect in App prints **2.**

Takeaway: `useEffect` is a React hook that runs code **after component renders**.

C) Mounting (run once)

3. Run effect only on first render (mount)

Use empty dependency array `[]`.

```
import { useEffect } from "react";

function App() {
  useEffect(() => {
    console.log("Mounted once ✅");
  }, []);

  return <h1>Hello</h1>;
}
```

Use cases

- initial API fetch
- read data from localStorage once
- start a timer once

D) Updating (run when something changes)

4. Run effect when a value changes

Put dependencies inside `[]`.

```
import { useEffect, useState } from "react";

function App() {
  const [count, setCount] = useState(0);

  useEffect(() => {
    console.log("Count changed:", count);
  }, [count]);

  return <button onClick={() => setCount(count +
1)}>{count}</button>;
}
```

Meaning: run this effect after render **whenever `count` changes**.

You can also run multiple values in dependency.

E) Unmounting (cleanup)

5. Cleanup function (runs on unmount)

If your effect sets up something that needs to be stopped, removed, or cleaned up, return a cleanup function. React runs that cleanup when the component unmounts, and also before the effect runs again because one of its dependencies changed.

```
useEffect(() => {
  console.log("Mounted");

  return () => {
    console.log("Unmounted (cleanup) ✅");
  };
}, []);
```

Use cases

- stop intervals

- remove event listeners
- cancel subscriptions

F) Practical Applications (must-know patterns)

6. Fetch data on mount (common)

```
import { useEffect, useState } from "react";

function Posts() {
  const [posts, setPosts] = useState([]);

  useEffect(() => {
    async function load() {
      let res = await
fetch("https://jsonplaceholder.typicode.com/posts");
      let data = await res.json();
      setPosts(data.slice(0, 5));
    }
    load();
  }, []);

  return (
    <ul>
      {posts.map(p => <li key={p.id}>{p.title}</li>)}
    </ul>
  );
}
```

Why inside function? `useEffect` callback should not directly be `async`.

Below is the wrong practice.

```
useEffect(async () => {

  }, []);
```

---

7. Timer with cleanup (interval)

```
import { useEffect, useState } from "react";

function Timer() {
  const [sec, setSec] = useState(0);

  useEffect(() => {
    let id = setInterval(() => {
      setSec(prev => prev + 1);
    }, 1000);

    return () => clearInterval(id); // cleanup
  }, []);

  return <h2>{sec}s</h2>;
}
```

---

8. Event listener with cleanup

```
import { useEffect, useState } from "react";

function WindowWidth() {
  const [w, setW] = useState(window.innerWidth);

  useEffect(() => {
    function handleResize() {
      setW(window.innerWidth);
    }

    window.addEventListener("resize", handleResize);
    return () => window.removeEventListener("resize", handleResize);
  }, []);

  return <h2>Width: {w}</h2>;
}
```

---

9. Sync state to localStorage

```
import { useEffect, useState } from "react";

function Notes() {
  const [text, setText] = useState(() =>
localStorage.getItem("note") || "");

  useEffect(() => {
    localStorage.setItem("note", text);
  }, [text]);

  return (
    <textarea value={text} onChange={(e) => setText(e.target.value)}
/>
  );
}
```

Common Beginner Mistakes

1. Forgetting dependency array → effect runs after every render

```
useEffect(() => { ... }); // runs after every render
```

2. Wrong dependencies causing infinite loop

```
useEffect(() => {
  setCount(count + 1);
}, [count]); // ❌ will loop forever
```

After component renders, effect runs => setCount changes state. State update triggers rerender and the effect runs again => setCount changes state. And this loops goes on and on.

3. Not cleaning intervals/listeners → keeps running even after component removed

4. Making `useEffect(async ()=>{})` directly. **Do not do this.**
   ✅ wrap async code inside.

Quick Practice (15 mins)

A) Mounting

1. On mount, fetch 5 users from:
   `https://jsonplaceholder.typicode.com/users`
   Render names in list.

B) Updating

2. Create a `search` state. On change of search, log:
   "Searching for: ___"

C) Unmounting

3. Create a `ShowTimer` button that toggles `<Timer />`.
   Ensure the timer stops when hidden (cleanup works).

D) Bug fix

4.

```
useEffect(() => {
  fetch(url).then(r => r.json()).then(setData);
}, [data]);
```

Why is this a problem? (Hint: dependency)